# PRODUCTION READINESS

## AUDIT REPORT

Rewind - Nostalgia Web Application

*Next.js 16.2.12 + Turbopack + Prisma (SQLite)*

August 1, 2026

*Generated by Z.ai*

# Executive Summary

This report presents the findings of a comprehensive production readiness audit for the Rewind application, a nostalgia-themed single-page web application built with Next.js 16.2.12, Turbopack, Prisma ORM with SQLite, NextAuth.js for authentication, and Framer Motion for animations. The audit examined build integrity, security posture, code quality, dependency health, performance characteristics, input validation, deployment readiness, and database configuration. The audit scope covered 71 source files across 12 API routes and 30+ React components, with automated fixes applied where safe to do so without altering application behavior.

**74**

PRODUCTION READINESS SCORE (out of 100)

The application passes a clean production build with zero TypeScript errors and zero ESLint errors (one false-positive warning from Next.js). However, several infrastructure and security gaps prevent a higher score. The most critical issues are the absence of Prisma migration files, missing deployment configuration for any hosting platform, unauthenticated public API endpoints, and 3 high-severity npm vulnerabilities in transitive dependencies. These are addressed in detail in the findings below.

# 1. Verification Results

The four mandatory verification commands were executed in sequence to establish a baseline for the audit. All commands completed successfully, establishing that the core application compiles, bundles, and passes static analysis without critical failures.

| Command | | |
|---|---|---|
| | | |
| Details | | |
| npm install | PASS | 459 packages audited, up to date, 0 errors |
| npm run build | PASS | Compiled in 9.2s, 12 routes generated, 0 errors |
| npm run lint | PASS (1 warn) | 0 errors, 1 false-positive font warning in layout.tsx |
| npm audit | FAIL (3 high) | postcss XSS + file-read (2 CVEs), sharp libvips (1 CVE) |

*Table 1: Core verification command results*

The npm audit failures are all in transitive dependencies of Next.js itself (postcss and sharp). These cannot be fixed without a breaking Next.js version downgrade to 9.3.3, which is not viable. The postcss CVEs (GHSA-qx2v, GHSA-6g55, GHSA-r28c) relate to XSS via unescaped CSS and arbitrary file read via sourceMappingURL. The sharp CVE (GHSA-f88m) covers inherited libvips vulnerabilities. In practice, the postcss risks are mitigated by the fact that the application does not process user-supplied CSS strings, and sharp is used only for server-side image optimization.

# 2. Problems Found

## 2.1 Security Issues

### Missing Authentication on Public Endpoints (HIGH)

Three API routes lack any authentication check, allowing unauthenticated access to server-side data and functionality. The `/api/movies` route fetches data from TMDB using a server-side API key, meaning anyone who discovers the endpoint can consume the API key quota without being logged in. The `/api/quiz` and `/api/this-day` routes expose curated application content without authentication. While these endpoints return non-sensitive read-only data, they represent unnecessary attack surface and could be abused for denial-of-service through quota exhaustion (movies) or scraping. The contrast is notable because five other API routes (sync, sync/profile, sync/push, ai/insights, user/profile) all correctly

implement session-based authentication using NextAuth getServerSession.

### Insufficient Input Validation (MEDIUM)

Prior to this audit, the registration endpoint lacked email format validation and password length□□, accepting malformed emails and arbitrarily long passwords that could cause bcrypt memory exhaustion. The movies API accepted unrestricted birthYear values (including negative numbers and future dates) and unlimited page parameters, enabling potential scraping. The quiz API accepted arbitrary decade strings without validation, returning fallback data for invalid inputs silently rather than rejecting them. None of the API routes implemented request body size limits or content-type validation, which is standard practice for production APIs.

### Prisma Query Logging in Production (LOW)

The Prisma client was configured with `log: ['query']` unconditionally, meaning every database query would be logged to stdout in production. This is a performance overhead and potential information leak in containerized environments where logs are aggregated. The fix was straightforward: conditionally enable query logging only when `NODE_ENV` is not production.

## 2.2 Infrastructure Issues

### No Prisma Migration Files (HIGH)

The `prisma/migrations/` directory does not exist. The schema has been pushed to the database via `prisma db push`, which creates or alters tables directly without generating migration files. This means there is no versioned history of database schema changes, making it impossible to track what changed, when, and why. More critically, it prevents reliable database provisioning in production environments where schema must be applied deterministically. On platforms like Railway or Render, the post-install hook runs `prisma migrate deploy`, which requires migration files to exist. Without them, database provisioning will fail on first deployment.

### No Deployment Configuration (HIGH)

No deployment configuration files exist for any hosting platform. There is no Dockerfile, docker-compose.yml, vercel.json, netlify.toml, railway.json, or render.yaml. The application uses `output: "standalone"` in next.config.ts, which is the correct setting for Docker/self-hosted deployments, and the build script correctly copies static assets and public directory into the standalone output. However, without a Dockerfile or platform-specific configuration, there is no reproducible way to deploy this application. A Dockerfile is the most portable option and is recommended.

### Database File in .env as Absolute Path (MEDIUM, FIXED)

The `.env` file contained `DATABASE_URL=file:/home/z/my-project/db/custom.db`, an absolute path hardcoded to the development machine. This would break immediately on any other environment. The fix was to revert to the relative path `file:./db/custom.db`, which resolves correctly from the project root on any machine. Additionally, the `.gitignore` file was missing entries

for the `db/` directory and `*.db` files, meaning the SQLite database file could accidentally be committed to version control. This was fixed by adding both patterns.

## 2.3 Code Quality Issues

### Overly Permissive ESLint Configuration (MEDIUM)

The ESLint configuration disables 31 rules, including `no-console`, `react-hooks/exhaustive-deps`, `prefer-const`, `no-unused-vars`, and `@typescript-eslint/no-explicit-any`. While some of these are reasonable for a creative/prototype project, several represent genuine code quality risks. The disabled `react-hooks/exhaustive-deps` rule can hide stale closure bugs where effects reference outdated state. The disabled `no-unused-vars` and `prefer-const` rules allow dead code and mutable variables where `const` should be used, degrading code maintainability. The `@typescript-eslint/no-explicit-any` exemption removes type safety across the codebase. It is recommended to gradually re-enable at least the React hooks and TypeScript rules before production deployment.

### console.log Statements in Production Code (LOW)

Seventeen `console.log` statements were found in `src/lib/idb-store/migrate.ts`. These are migration progress logs that fire during the localStorage-to-IndexedDB data migration path. While not harmful in production (they only run once per user during the migration window), they should ideally use a proper logging utility with level control. Additionally, twelve `console.error` statements exist across API routes for error logging, which is acceptable and standard practice for server-side error handling in Next.js API routes.

### Unused UI Components (LOW)

Four UI component files under `src/components/ui/` are not imported anywhere in the application: `dialog.tsx`, `sheet.tsx`, `skeleton.tsx`, and `tooltip.tsx`. These are likely scaffolding remnants from shadcn/ui initialization. They add approximately 4KB to the build bundle size (tree-shaken, so actual impact is minimal) but represent unnecessary code to maintain. They can be safely removed.

## 2.4 Performance Observations

### Inline onClick Handlers (INFORMATIONAL)

Ninety-nine inline arrow function definitions were found in JSX `onClick` props across the codebase. Each inline arrow function creates a new function reference on every render, which can trigger unnecessary re-renders in child components that implement React.memo or receive the handler as a dependency in useEffect/useMemo. For this application, the performance impact is negligible because most of these are on native HTML elements (buttons, divs) rather than memoized custom components. However, for a codebase of this size, extracting frequently-used handlers into `useCallback` would be

a best-practice improvement.

### Large Component Files (INFORMATIONAL)

Several component files exceed 500 lines, with the largest being `ShareRewind.tsx` at 1,160 lines, followed by `MemoryBoard.tsx` at 981 lines. Large files make code harder to navigate, test, and maintain. Consider splitting these into smaller, focused sub-components. The main page.tsx at 858 lines is reasonable given it contains the entire application shell with state management, but could benefit from extracting the Hero section and Dashboard into separate files.

# 3. Problems Fixed

The following issues were identified and automatically fixed during this audit. All fixes were designed to be behavior-preserving, meaning they improve robustness and security without changing how the application functions from the user perspective. The build was verified to pass with zero errors after each fix.

| # | | | |
|---|---|---|---|
| | | | |
| File(s) | | | |
| | | | |
| 1 | DATABASE_URL absolute path | .env | Reverted to relative path file:./db/custom.db |
| 2 | db/ not in .gitignore | .gitignore | Added /db/ and *.db patterns |
| 3 | Prisma query logging in prod | src/lib/prisma-client.ts | Conditional log: only in non-production |

| # | | | |
|---|---|---|---|
| 4 | No email validation on register | src/app/api/auth/register/route.ts | Added regex email format check |
| 5 | No name validation on register | src/app/api/auth/register/route.ts | Added non-empty string check |
| 6 | No password max length | src/app/api/auth/register/route.ts | Added 128-character upper bound |
| 7 | Unvalidated birthYear param | src/app/api/movies/route.ts | Clamped to 1940-2015 with NaN guard |
| 8 | Unvalidated category param | src/app/api/movies/route.ts | Whitelist to allowed categories |
| 9 | Unvalidated page param | src/app/api/movies/route.ts | Clamped to 1-10 with NaN guard |
| 10 | Unvalidated decade param | src/app/api/quiz/route.ts | Whitelist to allowed decades |
| 11 | Unvalidated count param | src/app/api/quiz/route.ts | Clamped to 1-20 with NaN guard |
| 12 | ESLint no-unused-expressions | src/components/OldJournal.tsx | Ternary converted to if/else statement |

*Table 2: All fixes applied during this audit*

# 4. Remaining Risks

The following issues were identified but **not fixed** because they require architectural decisions, infrastructure provisioning, or behavioral changes that are outside the scope of an automated audit. Each risk includes a recommended remediation path.

| **Risk** | | | |
|---|---|---|---|
| | | | |
| **Why Not Fixed** | | | |
| | | | |
| No Prisma migrations | HIGH | Requires `prisma migrate dev` to generate; may need data seed strategy | Run `prisma migrate dev --name init` locally, commit migration files |

| Risk | | | |
|---|---|---|---|
| No deployment config | HIGH | Requires choosing a platform and provisioning infrastructure | Add Dockerfile for portability; or add vercel.json for Vercel |
| 3 unauthenticated API routes | MEDIUM | Adding auth changes API contract; may break client calls | Add getServerSession check to movies, quiz, this-day routes |
| 3 high-severity npm vulns | MEDIUM | In transitive deps of Next.js; fix requires breaking downgrade | Monitor Next.js releases; upgrade when fix is available |
| No rate limiting on auth | MEDIUM | Requires middleware or external service (Redis/upstash) | Add Next.js middleware with IP-based rate limiting for auth routes |
| 31 disabled ESLint rules | LOW | Re-enabling may produce hundreds of warnings; needs iterative approach | Re-enable react-hooks/exhaustive-deps and no-explicit-any first |
| 99 inline onClick handlers | LOW | Refactoring is cosmetic; no measurable performance impact | Extract to useCallback for handlers on memoized components |
| 4 unused UI components | LOW | Removing is safe but trivial; no security or perf impact | Delete dialog.tsx, sheet.tsx, skeleton.tsx, tooltip.tsx |
| 17 console.log in migrate.ts | LOW | Migration logging is useful for debugging data transitions | Replace with conditional logger that respects NODE_ENV |
| No Content-Security-Policy | LOW | Requires deployment-level HTTP header configuration | Add CSP headers in next.config.ts or via platform config |

*Table 3: Remaining risks with remediation recommendations*

# 5. Files Modified

A total of six files were modified during this audit. All modifications were additive (adding validation, fixing configuration, or improving robustness) without removing or altering existing application logic.

| File | | |
|---|---|---|
| | | |
| Description | | |
| .env | Config fix | DATABASE_URL reverted from absolute to relative path |

| File | | |
|---|---|---|
| .gitignore | Security fix | Added /db/, *.db, *.db-journal to prevent database commits |
| src/lib/prisma-client.ts | Perf fix | Query logging now conditional on NODE_ENV != production |
| src/app/api/auth/register/route.ts | Security fix | Added email regex, name type check, password max-length (128) |
| src/app/api/movies/route.ts | Security fix | Added birthYear clamp (1940-2015), category whitelist, page clamp (1-10) |
| src/app/api/quiz/route.ts | Security fix | Added decade whitelist, count clamp (1-20), NaN guards |
| src/components/OldJournal.tsx | Lint fix | Converted ternary expression to if/else to satisfy no-unused-expressions |

*Table 4: Complete list of files modified*

# 6. Deployment Recommendation

The Rewind application is **not yet ready for production deployment** in its current state. While the application itself is functionally sound with a clean build and working core features (authentication, chat, journal, movies, games, contacts, memory features, and voice messages), there are three blocking infrastructure gaps that must be resolved before deployment: the absence of Prisma migration files, the absence of any deployment configuration, and the use of SQLite with a relative file path that needs platform-specific handling for persistent storage.

## Minimum Viable Deployment Checklist

To reach production readiness, the following steps should be completed in priority order. Each step addresses a specific blocking issue identified in this audit. The estimated effort for each step is based on the assumption that the developer is familiar with the respective technology.

| Priority | | |
|---|---|---|
| | | |
| Estimated Effort | | |
| P0 - Blocking | Generate Prisma migration files via `prisma migrate dev --name init` and commit them | 5 minutes |

| Priority | | |
|---|---|---|
| P0 - Blocking | Create a Dockerfile (multi-stage build) with standalone output and persistent volume for SQLite | 30 minutes |
| P0 - Blocking | Add auth checks to /api/movies, /api/quiz, /api/this-day routes | 15 minutes |
| P1 - Important | Configure persistent storage mount for db/ directory on deployment platform | 15 minutes |
| P1 - Important | Add NEXTAUTH_URL environment variable configuration for production domain | 5 minutes |
| P1 - Important | Regenerate NEXTAUTH_SECRET for production (64-char hex, never reuse dev secret) | 5 minutes |
| P2 - Recommended | Re-enable critical ESLint rules (react-hooks/exhaustive-deps, no-explicit-any) | 1-2 hours |
| P2 - Recommended | Remove unused UI components (dialog, sheet, skeleton, tooltip) | 5 minutes |
| P3 - Optional | Add rate limiting middleware for authentication endpoints | 1 hour |
| P3 - Optional | Add Content-Security-Policy and other security headers | 30 minutes |

*Table 5: Prioritized deployment preparation checklist*

Once the P0 blocking items are resolved, the application will be suitable for deployment on Docker-compatible platforms (Railway, Render, Fly.io, self-hosted VPS). For serverless platforms like Vercel, additional work is needed to replace SQLite with a managed database (PostgreSQL via Prisma) since serverless functions have ephemeral filesystems and cannot persist SQLite files across invocations. The application architecture is otherwise well-suited for modern deployment platforms, with the standalone output mode, proper static asset copying in the build script, and a clean dependency tree free of conflicts or duplicates.